

Bootcamp Databricks

Engenharia de Dados dentro da Plataforma Data & AI

I&D Capgemini Brasil, Setembro/2026

Agenda

Aulas	Assunto
Dia 1 – 14/09/2026	Fundamentos de Engenharia de dados e Databricks
Dia 2 – 15/09/2026	SQL, bancos, tabelas e Delta Lake
Dia 3 – 16/09/2026	Unity Catalog, schemas, volumes e governança
Dia 4 – 17/09/2026	Ingestão de dados: batch, incremental, Auto Loader
Dia 5 – 18/09/2026	ETL vs ELT & transformação arquitetura medalhão
Dia 6 – 21/09/2026	Modelagem de dados para Analytics
Dia 7 – 22/09/2026	Processamento distribuído com Spark no Databricks
Dia 8 – 23/09/2026	Streaming e processamento contínuo
Dia 9 – 24/09/2026	Orquestração, Jobs, DataOps e boas práticas
Dia 10 – 25/09/2026	Qualidade, observabilidade, arquitetura e desafio final

Dia 4 - Ingestão de dados: batch, incremental, Auto Loader

Bootcamp Databricks

I&D Capgemini Brasil, Setembro/2026





Objetivos do Dia 4

O aluno vai sair sabendo escolher e executar padrões básicos de ingestão.

- Explicar fontes de dados e padrões de entrada no Lakehouse.
- Diferenciar full load, incremental load, CDC e streaming/Auto Loader.
- Entender quando usar COPY INTO e quando usar Auto Loader.
- Criar uma ingestão para Bronze a partir de arquivos em volume.
- Configurar checkpoint, schema location e validação de carga.

Mensagem central

Ingestão não é só carregar arquivo. É controlar estado, schema, idempotência, rastreabilidade e destino.

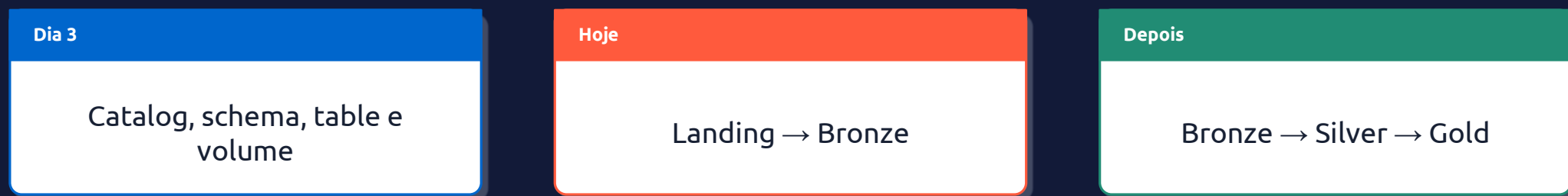
Entregável

Pipeline Bronze com ingestão automatizada e validada.



Recap do Dia 3: governança antes da ingestão

O pipeline começa melhor quando o destino já tem estrutura e dono.



- O volume landing do Dia 3 representa a fonte governada.
- A tabela Bronze representa persistência auditável do dado bruto.
- Checkpoint e schema location representam controle operacional da ingestão.
- O namespace completo evita ambiguidade no laboratório.

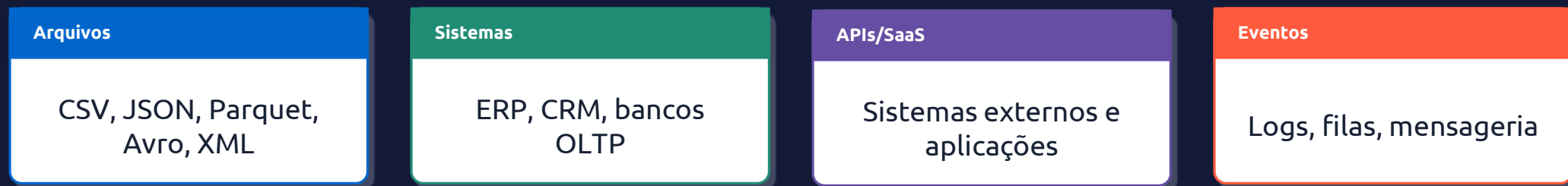


Parte 1 - Padrões de ingestão



Fontes de dados: de onde os dados chegam?

A forma de ingestão depende da fonte, frequência, volume e criticidade.

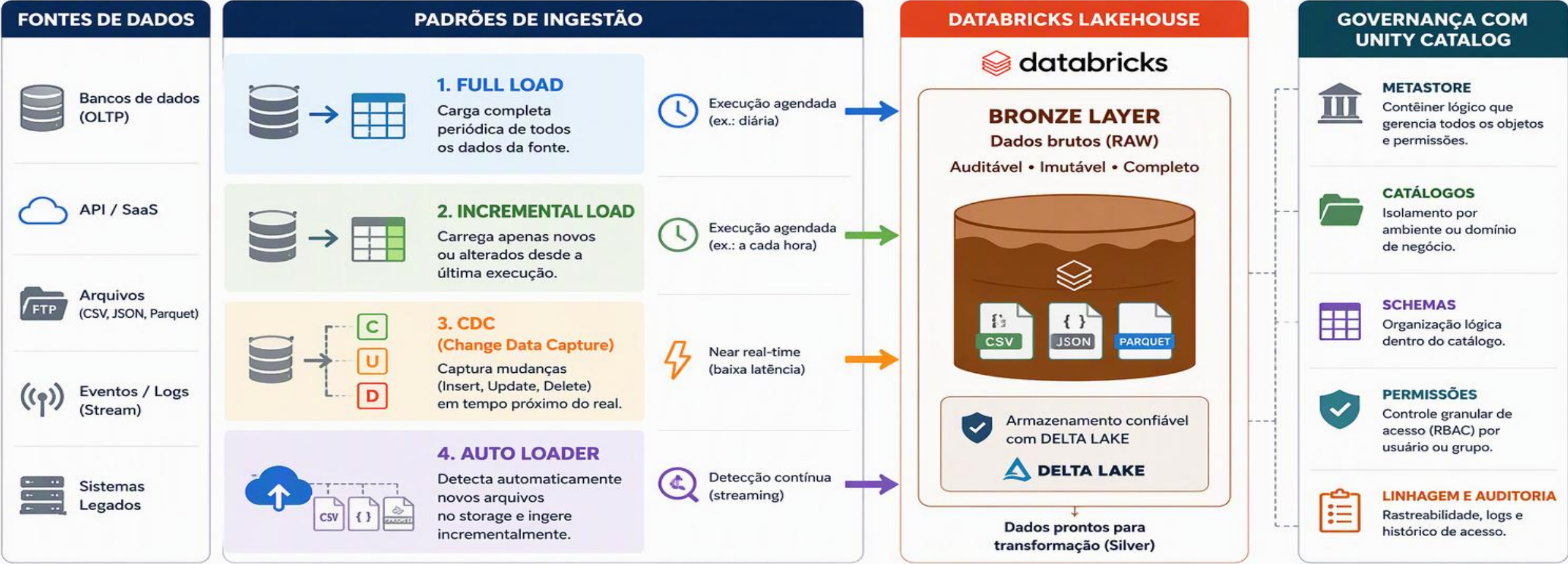


Perguntas de engenharia antes de escolher a técnica

- O dado chega em lote ou continuamente?
- Preciso reprocessar todo o histórico ou só novidades?
- Como detectar mudanças?
- Como tratar schema drift e arquivos ruins?
- Onde ficará o estado da ingestão?

PADRÕES DE INGESTÃO DE DADOS NO DATABRICKS

Todos os dados entram na camada BRONZE (raw) de forma segura, escalável e governada





Full load vs Incremental load

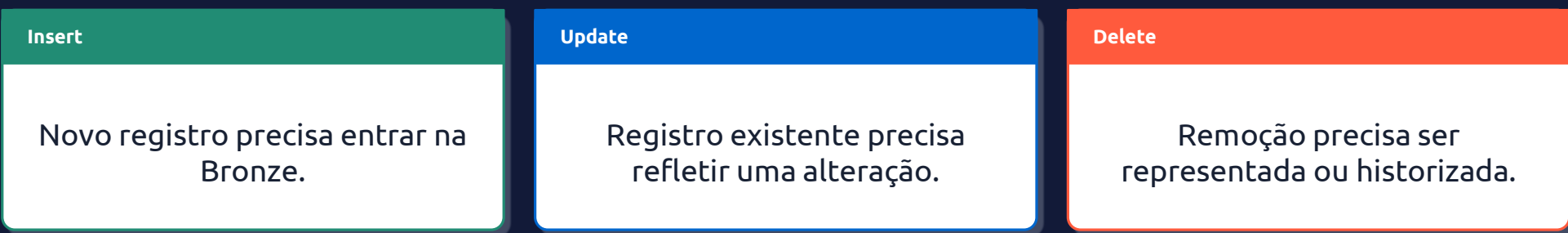
A primeira decisão é entre simplicidade operacional e eficiência incremental.

Critério	Full load	Incremental load
Como funciona	Recarrega a base completa	Carrega apenas novos/alterados
Vantagem	Mais simples de entender e reconstruir	Reduz processamento e custo
Atenção	Pode ficar caro e demorado em bases grandes	Exige controle de estado, chave ou watermark
Uso didático	Primeira carga ou tabelas pequenas	Rotina recorrente e arquivos novos



CDC: Change Data Capture

CDC é a ponte entre sistemas transacionais e pipelines analíticos consistentes.



- CDC exige coluna de operação, timestamp, sequência ou log de origem.
- Em Bronze, preserve o máximo possível da mudança original.
- Tratamento de merge e regras de negócio virão com mais força no Dia 5.
- Não confunda CDC com simples ingestão de arquivo novo.



Parte 2 - COPY INTO e Auto Loader



COPY INTO: carga SQL idempotente

Opção simples para carregar arquivos de uma localização para tabela Delta.

Quando usar

Cargas batch simples, SQL-first, arquivos em cloud storage ou volumes.

Ponto forte

Arquivos já carregados são ignorados em execuções subsequentes.

Atenção

Para este laboratório, COPY INTO é a forma mais simples de demonstrar ingestão incremental sem duplicar arquivos já processados.

```
CREATE TABLE IF NOT EXISTS capgemini_academy.bronze.flight_data;
```

```
COPY INTO capgemini_academy.bronze.flight_data  
FROM '/Volumes/capgemini_academy/landing/files/flight'  
FILEFORMAT = JSON  
FORMAT_OPTIONS ('multiLine' = 'false')  
COPY_OPTIONS ('mergeSchema' = 'true');
```

Auto Loader: ingestão incremental de arquivos



AUTO LOADER

Internals & Flow



1 DISCOVER

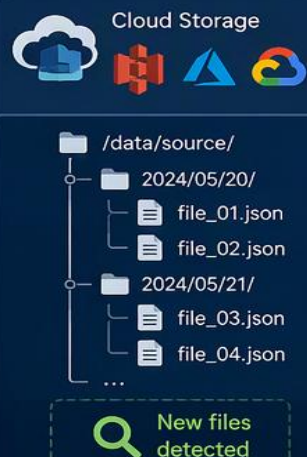
2 INGEST

3 TRACK

4 EVOLVE

5 COMMIT

1 FILE DISCOVERY



2 AUTO LOADER SOURCE

cloudFiles



3 CHECKPOINT & METADATA



4 SCHEMA EVOLUTION



5 DELTA TARGET TABLE



UNITY CATALOG | GOVERNANCE & SECURITY



Centralized Metastore



Fine-grained Access Control



Data Lineage



Audit Logs



Data Quality



Classification



Cross-workspace Sharing



Como o Auto Loader controla estado

Checkpoint e schema location são parte da operação, não detalhe técnico.

cloudFiles

Fonte Structured Streaming usada pelo Auto Loader.

Checkpoint

Guarda progresso e permite retomada em falhas.

Schema location

Armazena inferência e evolução de schema.

- A documentação informa que metadados dos arquivos descobertos são persistidos no checkpoint.
- Esse estado ajuda a garantir processamento exatamente uma vez ao escrever em Delta Lake.
- Não armazene checkpoint/schema sob o diretório da tabela em Unity Catalog.



Auto Loader com Unity Catalog

Segurança e organização continuam valendo durante a ingestão.

- Auto Loader pode ingerir dados com segurança a partir de external locations configuradas com Unity Catalog.
- A documentação recomenda armazenar checkpoint e evolução de schema em locais gerenciados pelo Unity Catalog.
- Unity Catalog não permite aninhar checkpoint ou arquivos de schema inference/evolution sob o diretório da tabela.
- Directory listing mode é suportado por padrão no Databricks Runtime 11.3 LTS e acima.

```
checkpoint_path = '/Volumes/capgemini_academy/landing/files/metadata/_flight_checkpoint'  
  
schema_path     = '/Volumes/capgemini_academy/landing/files/metadata/_flight_schema'  
  
source_path     = '/Volumes/capgemini_academy/landing/files/flight'
```

```
checkpoint_path = onde o Auto Loader controla progresso  
schema_path     = onde o Auto Loader controla schema  
source_path     = onde chegam os arquivos  
target_table    = tabela Delta Bronze
```



COPY INTO vs Auto Loader: decisão prática

Não escolha pela moda. Escolha pelo requisito operacional.

Cenário	Escolha sugerida
Carga batch simples e SQL-first	COPY INTO
Arquivos novos chegando continuamente	Auto Loader
Necessidade de checkpoint explícito	Auto Loader
Reexecução idempotente com arquivos já processados	COPY INTO ou Auto Loader
Produção com incrementalidade robusta	Auto Loader em Lakeflow pipelines ou job trigger

Comece pelo requisito: frequência, volume, tolerância a falha, schema drift, governança e custo.



Parte 3 - Demo e laboratório

Objetivo: Gerar dados de amostra em volume governado e testar padrões de ingestão para Bronze: full load, COPY INTO incremental/idempotente, Auto Loader com checkpoint/schema location e CDC preservado para transformação posterior.

Narrativa: no Dia 3 organizamos o ambiente governado. No Dia 4 usamos esse desenho para carregar dados com estado, schema, rastreabilidade e validação.

Jornada prática do laboratório

1 Preparar ambiente

SET

Criar metadata volume, pastas source/checkpoint/schema e tabelas Bronze.

2 Gerar lote inicial

JSON

Criar arquivos JSON fictícios de pedidos dentro da landing.

3 Full load

FULL

Ler todos os arquivos e sobrescrever a tabela pedidos_full_load.

4 COPY INTO

COPY

Carregar arquivos com reexecução idempotente.

5 Novo lote

INC

Adicionar novos arquivos na landing para simular incrementalidade.

6 Auto Loader

AUTO

Processar arquivos com cloudFiles, checkpoint e schema location.

7 Schema drift

SCH

Adicionar nova coluna e observar evolução de schema controlada.

8 CDC raw

CDC

Preservar inserts, updates e deletes em Bronze para o Dia 5.

Estrutura esperada

```
academy_<seu_nome>
├── landing
│   ├── files
│   │   ├── pedidos
│   │   └── metadata
│   │       ├── _checkpoint
│   │       └── _schema
│   └── bronze
│       ├── pedidos_full_load
│       ├── pedidos_copy_into
│       ├── pedidos_autoloader
│       └── pedidos_cdc_raw
```

Landing → Bronze

Checklist de validação

- ✓ Arquivos gerados na landing
- ✓ Full load executado
- ✓ COPY INTO sem duplicidade
- ✓ Auto Loader com checkpoint
- ✓ Schema drift observado
- ✓ CDC raw preservado em Bronze

Comandos e caminhos centrais

```
source_path = /Volumes/.../landing/files/pedidos
checkpoint_path = /Volumes/.../metadata/_checkpoint
schema_path = /Volumes/.../metadata/_schema
COPY INTO bronze.pedidos_copy_into
readStream.format("cloudFiles")
.trigger(availableNow=True).toTable(...)
```

Ingestão bem feita não apenas move dados. Ela cria confiança para tudo que será transformado depois.



Demo guiada: carregar arquivos para Bronze

LABORATÓRIO – DIA 3

Auto Loader + Unity Catalog

Objetivo: Ingerir arquivos JSON de um volume (landing) para uma tabela Delta (Camada Bronze)



Arquivos no volume (landing) → Auto Loader (Structured Streaming) → Delta Lake (Camada Bronze) → Tabela Gerenciada no Unity Catalog

1 Volume Landing (Fonte)

Catalog Explorer

Volumes > main > bronze_landing > json

[Upload](#) [+ Folder](#) [Refresh](#)

Name

- _checkpoint
- pedido_2024-07-24_01.json
- pedido_2024-07-24_02.json
- pedido_2024-07-24_03.json
- pedido_2024-07-24_04.json

Caminho do volume

/Volumes/main/bronze_landing/json

2 Configurações do Auto Loader

Formato	JSON
Modo de detecção	Directory listing (padrão)
Checkpoint	/Volumes/main/bronze_landing/_checkpoint/pedidos
Schema Evolution	Habilitada
Trigger	Available Now (micro-batch)

3 Notebook – Auto Loader para Bronze

```
from pyspark.sql.functions import *

checkpoint_path = "/Volumes/main/bronze_landing/_checkpoint/pedidos"
source_path = "/Volumes/main/bronze_landing/json"

(spark.readStream
  .format("cloudFiles")
  .option("cloudFiles.format", "json")
  .option("cloudFiles.schemaLocation", checkpoint_path)
  .load(source_path)
)

.writeStream
  .option("checkpointLocation", checkpoint_path)
  .trigger(availableNow=True)
  .toTable("main.bronze.pedidos_bronze")
```

✓ Comando concluído Linhas processadas: 150 Novos arquivos: 3 Duração: 00:00:18

4 Resultado – Tabela Bronze (Unity Catalog)

main > bronze > pedidos_bronze Delta Table

pedido_id	cliente_id	produto_id	quantidade	valor_total	data_pedido	_ingested_at
1001	C001	P100	2	59.90	2024-07-24T10:15:00Z	2024-07-24 10:15:23
1002	C002	P200	1	89.50	2024-07-24T10:16:00Z	2024-07-24 10:15:23
1003	C001	P300	3	149.70	2024-07-24T10:17:00Z	2024-07-24 10:15:23
1004	C003	P100	1	29.90	2024-07-24T10:18:00Z	2024-07-24 10:15:23
1005	C004	P400	5	249.50	2024-07-24T10:19:00Z	2024-07-24 10:15:23

150 rows 7 columns

5 Checklist do Laboratório

- ✓ Acessei o volume de landing
- ✓ Verifiquei os arquivos JSON
- ✓ Execução do notebook Auto Loader
- ✓ Tabela Bronze criada no Unity Catalog
- ✓ Dados carregados com sucesso
- ✓ Consulta SQL executada e validada
- ✓ Entendi checkpoint e schema evolution

O que aprendemos

- Como Unity Catalog organiza e governa nossos dados.
- Volumes como camada controlada de arquivos.
- Auto Loader para ingestão incremental confiável.
- Tabela Delta Gerenciada no catálogo.
- Primeira camada do pipeline (Bronze) operacional.

Dica do Instrutor

No Dia 4 vamos aprofundar ingestão incremental, evolução de schema e estratégias de qualidade dos dados.

Ambiente: DEV Catálogo: main Schema: bronze Tabela: pedidos_bronze Camada: BRONZE

- Criar volume de metadata para checkpoint/schema.
- Usar arquivos do volume landing criado no Dia 3, ou gerar novos pelo notebook Dia 4.
- Executar Auto Loader com trigger availableNow.
- Validar registros carregados na tabela Bronze.

Entregável da demo

Tabela capgemini_academy.bronze.pedidos_*
criada e consultável.



Script demo: preparação do ambiente

```
1 spark.sql(f"USE CATALOG {catalog_name}")
2 spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog_name}.{landing_schema}")
3 spark.sql(f"CREATE SCHEMA IF NOT EXISTS {catalog_name}.{bronze_schema}")
4 spark.sql(f"CREATE VOLUME IF NOT EXISTS {catalog_name}.{landing_schema}.{landing_volume}")
5 spark.sql(f"CREATE VOLUME IF NOT EXISTS {catalog_name}.{landing_schema}.{metadata_volume}")
6
7 dbutils.fs.mkdirs(source_path)
8 dbutils.fs.mkdirs(checkpoint_path)
9 dbutils.fs.mkdirs(schema_path)
10
11 print("Ambiente preparado.")
```



Laboratório - Parte 1: criar dados de exemplo

Se não houver arquivo disponível, gere JSON no próprio volume.

```
1 orders_batch_001 = [  
2     make_order(1001, "C001", "P100", 2, 59.90, 15, status="novo"),  
3     make_order(1002, "C002", "P200", 1, 89.50, 16, status="novo"),  
4     make_order(1003, "C003", "P300", 3, 149.70, 18, status="novo"),  
5     make_order(1004, "C001", "P400", 1, 39.90, 20, status="novo"),  
6     make_order('null', "C003", "P600", -5, 99.90, 20, status="novo"),  
7 ]  
8  
9 write_jsonl(f"{source_path}/batch_001.json", orders_batch_001)  
10 list_landing()
```



Laboratório - Parte 2: executar ingestão incremental

Você deve rodar Auto Loader e validar a primeira carga.

```
1 query = (spark.readStream
2     .format("cloudFiles")
3     .option("cloudFiles.format", "json")
4     .option("cloudFiles.schemaLocation", schema_path)
5     .option("cloudFiles.schemaEvolutionMode", "addNewColumns")
6     .load(source_path)
7     .withColumn("_source_file", col("_metadata.file_path"))
8     .withColumn("_ingested_at", current_timestamp())
9     .writeStream
10    .option("checkpointLocation", checkpoint_path)
11    .option("mergeSchema", "true")
12    .trigger(availableNow=True)
13    .toTable(table_auto))
14
15 query.awaitTermination()
16 spark.sql(f"SELECT COUNT(*) AS qtd_linhas FROM {table_auto}").show()
17 display(spark.table(table_auto).orderBy("pedido_id"))
```



Laboratório - Parte 3: testar evolução simples de schema

Adicionar nova coluna permite discutir schema drift.

```
1 orders_batch_003_schema = [  
2     **make_order(1007, "C006", "P600", 2, 179.80, 42, channel="app", status="novo"), "cupom_desconto": "APP10"},  
3     **make_order(1008, "C007", "P700", 1, 299.90, 45, channel="web", status="novo"), "cupom_desconto": "WEB20"},  
4 ]  
5  
6 write_jsonl(f"{source_path}/batch_003_schema_drift.jsonl", orders_batch_003_schema)  
7 list_landing()
```

Discussão

O objetivo é observar o impacto de um campo novo e conectar com governança de schema.



Validação: ingestão sem validação é só movimento

Finalize o laboratório com evidências objetivas.

```
1 validation_queries = [  
2     f"SELECT 'full_load' AS tabela, COUNT(*) AS linhas FROM {table_full}",  
3     f"SELECT 'copy_into' AS tabela, COUNT(*) AS linhas FROM {table_copy}",  
4     f"SELECT 'autoloader' AS tabela, COUNT(*) AS linhas FROM {table_auto}",  
5     f"SELECT 'cdc_raw' AS tabela, COUNT(*) AS linhas FROM {table_cdc_raw}",  
6 ]  
7  
8 for q in validation_queries:  
9     spark.sql(q).show()  
10  
11 spark.sql(f"DESCRIBE TABLE EXTENDED {table_auto}").show(truncate=False)
```

- Verifique contagem de linhas.
- Verifique colunas esperadas.
- Verifique se os arquivos novos não foram duplicados.
- Explique onde estão fonte, checkpoint, schema e destino.



Quiz rápido de fixação

- 1) Quando você usaria COPY INTO em vez de Auto Loader?
- 2) Por que checkpoint não deve ser tratado como pasta temporária?
- 3) Qual a diferença entre full load e incremental load?
- 4) O que muda quando um novo campo aparece no arquivo?
- 5) Por que Bronze deve preservar o dado bruto?

Resposta esperada: você deve explicar o raciocínio, não apenas decorar comandos.



Erros comuns em ingestão

Ponto provocativo para formar bons hábitos desde o começo.

Erro 1

Usar produção como fonte de laboratório.

Erro 2

Não separar landing, checkpoint, schema e tabela.

Erro 3

Criar Bronze já filtrando demais o dado.

Erro 4

Não validar duplicidade ou reexecução.

Erro 5

Tratar schema evolution como detalhe menor.

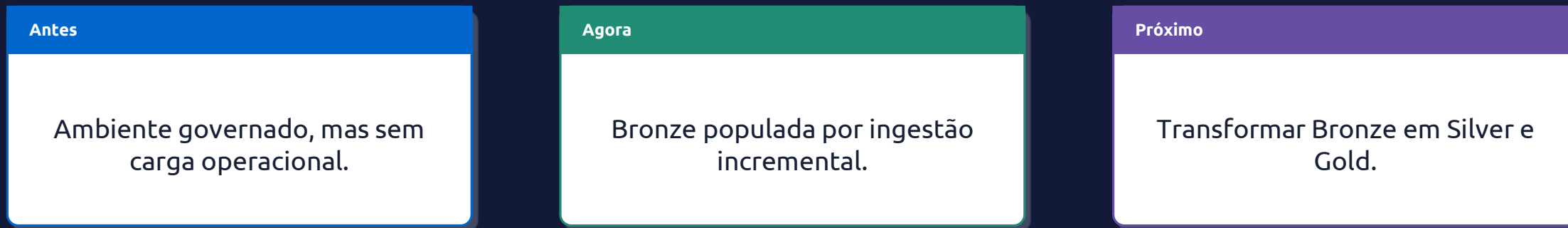
Erro 6

Não documentar fonte, frequência e owner.



Fechamento: o que muda depois do Dia 4

Agora existe dado bruto confiável para transformar.



Dia 5: ETL vs ELT e transformação Bronze / Silver / Gold.

Vamos limpar, padronizar, deduplicar e aplicar regras de negócio nos dados ingeridos hoje



Desafio do Dia 4 – Assistir e resumir

- Comando COPY INTO do Databricks

<https://www.youtube.com/watch?v=sjItHOjZsY4>

- AUTO LOADER no Databricks

https://www.youtube.com/watch?v=g7d1U2_dWS8

Thank you.

Make it real.

Capgemini 